

**BHASKARACHARYA NATIONAL INSTITUTE FOR
SPACE APPLICATIONS AND GEO-INFORMATICS**
WEEKLY PROGRESS REPORT (08/06/2026 – 14/06/2026)

WEEK 4

Project Name

OSINT Collection from Social Media Network

PROJECT DESCRIPTION :

Continuation of the linkmap OSINT pipeline. Week 4 focused entirely on the design and development of the Nexus — a new dedicated interactive graph explorer page featuring a full-viewport D3.js force-directed network, a split-panel layout with a dark graph canvas and an off-white detail panel, dual PostgreSQL and Neo4j data integration, and deep per-handle analytics accessible directly from the graph.

GROUP MEMBER :

Rishav Pal, Sayantan Mandal, Shivam Kumar

GROUP ID :

2026G268

GROUP GUIDE :

Dr. JHUMMARWALA ABDUL

GROUP COORDINATOR :

Dr. JHUMMARWALA ABDUL

COLLEGE GUIDE NAME :

Dr. SHWETA SAHARAN

COLLEGE GUIDE No. :

9509676076

COLLEGE GUIDE EMAIL. :

SHWETA.SAHARAN@GSV.AC.IN

COLLEGE NAME :

GATI SHAKTI VISHWAVIDYALAYA

WEEK 4 — DAILY WORK LOG

Day 1–2 | 08–09 June 2026 | Nexus Backend — API Design & Data Architecture

The first two days were dedicated to designing and building the complete backend for the Nexus page — a new interactive graph explorer that merges PostgreSQL and Neo4j data into a unified handle-centric intelligence view.

- Designed the Nexus data architecture: the page requires a three-tier data fetch — (1) graph seed data (all handles with aggregate stats) for initial rendering, (2) per-handle deep details (profile, hashtags, mentions, links, topics, tweet stats, most liked tweet) for the side panel, and (3) Neo4j cross-handle relationship data (who mentions this handle, which handles share hashtags with it) for graph edge rendering.
- Created backend/routes/nexus.py (462 lines) with a dedicated run_pg_query() helper that uses the shared ThreadedConnectionPool from postgres_client.py via pool.getconn() / pool.putconn() — fully consistent with the pooled pattern established in Week 3.
- Built GET /api/nexus/graph — returns all scraped handles with aggregate statistics computed via 4 correlated LEFT JOIN subqueries in a single SQL call: tweet count, unique hashtag count, unique mention count, and unique link count per handle. Uses LATERAL unnest() and jsonb_array_elements() for efficient array column aggregation.
- Built GET /api/nexus/handle/{handle}/tweets — returns all tweets for a given handle ordered by timestamp DESC. Includes handle validation (regex pattern), 404 check against profiles table before querying tweets table, and full tweet field serialisation including JSONB links array and timestamp isoformat conversion.
- Built GET /api/nexus/handle/{handle}/details — the most data-rich endpoint in the project. Runs 7 PostgreSQL queries in sequence: profile with aggregated stats, top 20 hashtags by frequency, top 20 mentions by frequency, top 20 links by share count, topic distribution, tweet engagement stats (total likes, retweets, avg likes), and most liked tweet. Merges the PostgreSQL data with Neo4j graph data (mentioned_by handles, co-hashtag handles) from get_neo4j_connections() into a single response object.
- Implemented get_neo4j_connections() as a graceful fallback function — if Neo4j is offline or the Cypher query fails, it logs a warning and returns empty lists rather than crashing. This ensures the Nexus page still renders with full PostgreSQL data even when the graph database is temporarily unavailable.
- Registered nexus.router in backend/server.py with prefix="" since the router already uses prefix="/api/nexus". Added GET /nexus.html route. Added nexus import to the master route import line.

Day 3 | 10 June 2026 | Nexus Frontend — Split-Panel Layout & D3.js Graph

Day 3 built the Nexus frontend HTML structure and the core D3.js force-directed graph rendering engine.

- Designed the Nexus split-panel layout in frontend/nexus.html: a full-viewport container (calc(100vh - nav-height)) divided into a 70% dark left panel (graph canvas) and 30% off-white right panel (detail panel). The left panel uses background-color: #0f172a (dark navy) with a subtle radial dot grid pattern matching Neo4j Browser aesthetics. The right panel uses the standard page background (#f0ede8) with a 2px black left border.
- Built a floating graph toolbar inside the left panel positioned top-right: contains Show Labels toggle, Node Size selector (followers / tweet count / equal), Filter by Topic dropdown, Zoom In / Zoom Out / Reset buttons, and a Refresh Graph button. Toolbar uses glassmorphism styling (rgba(15,23,42,0.9) background, backdrop-filter blur) with white monospace text on the dark canvas.
- Implemented the right panel with three states: placeholder (no node selected, shows "Click any node to explore"), loading skeleton (animating shimmer while data fetches), and populated detail view (full handle intelligence breakdown). Added a Close button to dismiss the panel and return to placeholder state.
- Built initSimulation() in frontend/js/nexus.js — D3 force simulation with: forceLink (distance varies by node level — handle-to-tweet links shorter, cross-handle links longer), forceManyBody with strength varying by node type, forceCenter, and forceCollide for overlap prevention.
- Defined a 5-level node hierarchy: Level 0 = scraped handle nodes (largest, white fill on dark canvas), Level 1 = hashtag nodes (cyan #06b6d4), Level 2 = mention nodes (purple #a855f7), Level 3 = link/domain nodes (orange #f97316), Level 4 = topic nodes (green #22c55e). Node radius scales with the selected size metric.
- Implemented buildBaseGraph() to construct the initial graph from the /api/nexus/graph response: creates one node per scraped handle, pre-positions them in a circle for stable initial layout, and initialises empty link arrays ready for expansion.

WEEK 4 — DAILY WORK LOG (continued)

Day 4 | 11 June 2026 | Nexus — Node Expansion, Interactions & Side Panel

Day 4 implemented the interactive expansion system — clicking a handle node on the graph fetches and expands its connected entities — and built the detail side panel.

- Implemented the node expansion pattern using `handleCache` (Map) and `tweetCache` (Map) for client-side caching so repeated clicks on the same node do not make duplicate API requests. `expandedHandles` (Map) tracks which handles are currently expanded on the graph. `expandedCategories` (Map) tracks which entity category (hashtags, mentions, links, topics) is expanded per handle.
- Built `expandHandle(handle)` — on first click: fetches `/api/nexus/handle/{handle}/details`, caches the result, creates child nodes for top 10 hashtags, top 10 mentions, top 10 links, and all topics, adds edges from the handle node to each child, restarts the simulation to incorporate new nodes. On second click (already expanded): collapses all child nodes and removes their edges, shrinks the graph back to handle-only view.
- Built the `expandCategory(handle, category)` function for granular expansion — individual category buttons (Hashtags, Mentions, Links, Topics) in the side panel allow expanding only one entity type at a time rather than all at once.
- Implemented node hover tooltips: on mouseover a positioned tooltip div shows handle name, node type, and key metric (follower count for handles, usage count for hashtags). Tooltip follows cursor position and hides on mouseout.
- Built the full side panel detail view rendered by `showSidePanel(handle, data)`: displays profile header (handle, display name, verified badge, follower/following counts), engagement stats (total tweets, total likes, total retweets, avg likes), topic pills, top hashtags with frequency bars, top mentions with frequency bars, top links list, most liked tweet card, and Neo4j connections section (who mentions this handle, co-hashtag handles). All rendered as pure HTML strings injected into the side panel content div.
- Added category expand buttons inside the side panel — clicking "Expand Hashtags", "Expand Mentions" etc. triggers `expandCategory()` to add only that entity type to the graph without expanding all categories simultaneously.
- Implemented `showToast(message, isError)` — a transient notification banner that appears bottom-center of the graph canvas for 3 seconds for user feedback on actions (node expanded, graph refreshed, error loading data).

Day 5–6 | 12–13 June 2026 | Nexus — Graph Controls, Filtering & Polish

Days 5 and 6 completed the Nexus graph toolbar functionality, node filtering, zoom controls, and visual polish.

- Implemented Show Labels toggle — when enabled, renders handle names as white text labels below each node on the dark canvas using D3 text elements with a semi-transparent dark background rect for legibility. Labels only show for Level 0 (handle) nodes by default; child node labels appear on hover only.
- Implemented Node Size selector with three modes: (1) By Followers — node radius scales logarithmically with `followers_count` (min 8px, max 30px), giving immediate visual weight to high-follower accounts; (2) By Tweet Count — radius scales with `tweet_count`; (3) Equal — all nodes render at a fixed radius for unbiased layout. Changing the selector immediately updates all node radii and restarts the simulation.
- Implemented Filter by Topic dropdown — populated from all topics available in the Nexus graph data. Selecting a topic dims all handle nodes whose topics array does not include the selected topic (opacity 0.15), highlights matching handles (opacity 1.0, slightly larger radius), and collapses all expanded non-matching handle subgraphs. Selecting "All Topics" restores normal opacity for all nodes.
- Implemented zoom controls using `d3.zoom()`: Zoom In / Zoom Out buttons call `svg.transition().call(zoom.scaleBy, 1.3)` and `0.7` respectively. Reset button calls `resetZoom()` which animates back to `scale(1) translate(0,0)`. The graph also supports mouse wheel zoom and drag-to-pan natively via `d3.zoom()`.
- Implemented Refresh Graph button — calls `loadGraph()` which re-fetches `/api/nexus/graph`, clears all expanded nodes and cached data, and rebuilds the base graph from fresh data. Used after a new scrape job completes.
- Added the graph auto-refresh listener pattern from the Network page — `listenForGraphUpdates()` polls `localStorage` every 5 seconds for an `activeScraperJob`. When `graph_ready` event is detected, shows the toast notification "New scrape data available — click Refresh" instead of the banner used on the Network page, since the Nexus toolbar already has a dedicated Refresh button.
- Added nav link "Nexus" to all 11 HTML files. Updated `frontend/js/nav.js` router to import `initNexus` and map it to the `nexus.html` page path.

Day 7 | 14 June 2026 | Integration Testing & Final Codebase Review

The final day was dedicated to end-to-end integration testing of the Nexus page against live data, final codebase review, and documentation updates.

- Ran full integration test of the Nexus page against live PostgreSQL and Neo4j data: verified `/api/nexus/graph` returns all 20 scraped handles with correct aggregate stats, `/api/nexus/handle/{handle}/details` returns complete merged data for all 20 handles, Neo4j fallback behaviour confirmed (page renders correctly when Neo4j is offline).
- Tested all graph interactions: node click expands correct child nodes, category expand buttons add only selected entity type, collapse removes child nodes and edges, tooltip appears on hover, side panel renders full data, close button dismisses panel.
- Tested all toolbar controls: Show Labels correctly toggles text elements, all three Node Size modes update radii correctly, Topic filter dims/highlights correct handles, Zoom In/Out/Reset animate smoothly, Refresh Graph clears cache and reloads.
- Confirmed `handleCache` and `tweetCache` prevent duplicate API requests — clicking an already-expanded handle reads from cache with zero network requests.
- Verified that the Nexus page correctly uses the shared PostgreSQL connection pool — no raw `psycopg2.connect()` calls, all connections returned to pool in `finally` blocks.
- Reviewed and confirmed all Week 3 fixes remain intact in the final codebase: `lifespan` context manager, `asyncio.get_running_loop()`, `write_tweets_batch()` in `main.py`, `_get_pool_conn()/_release_conn()` in `postgres_writer.py`, `lxml` HTML parser in `parser.py`.
- Updated `README.md` with Nexus page documentation: endpoint descriptions, interaction guide, and architecture note on the dual-database merge pattern.

WEEK 4 — MEMBER CONTRIBUTIONS

Rishav Pal — Data Engineer

- Designed the Nexus three-tier data architecture: graph seed data layer (PostgreSQL aggregate JOIN), per-handle details layer (7 PostgreSQL queries merged with Neo4j), and client-side caching layer (handleCache + tweetCache Maps).
- Built backend/routes/nexus.py (462 lines) with run_pg_query() pool helper, get_neo4j_connections() with graceful fallback, and all three API endpoints.
- Designed and implemented the 4-correlated LEFT JOIN SQL query in GET /api/nexus/graph using LATERAL unnest() and jsonb_array_elements() for efficient array column aggregation in a single database round trip.
- Implemented GET /api/nexus/handle/{handle}/details — 7 sequential PostgreSQL queries plus Neo4j data merged into a single JSON response covering profile, hashtags, mentions, links, topics, stats, most liked tweet, and cross-handle graph connections.
- Registered nexus.router in backend/server.py, added /nexus.html FileResponse route, and confirmed all Week 3 fixes (lifespan, get_running_loop, batch writes) remain intact.
- Conducted full integration testing of all three Nexus endpoints against live data, verified Neo4j fallback behaviour, and confirmed shared pool usage in all queries.

Sayantana Mandal — OSINT Analyst

- Designed the Nexus split-panel layout in frontend/nexus.html: 70/30 viewport split, dark navy left panel with dot-grid background, off-white right panel with 2px border, full-height overflow-hidden container.
- Built the floating glass-morphism graph toolbar with all five control groups: Show Labels toggle, Node Size selector (3 modes), Topic filter dropdown, Zoom controls (In/Out/Reset), and Refresh Graph button.
- Designed and implemented the three-state right detail panel: placeholder, loading skeleton (shimmer animation), and populated view with profile header, engagement stats, topic pills, frequency bars, top links, most liked tweet card, and Neo4j connections section.
- Implemented showToast() notification system for user feedback on graph actions (node expanded, collapsed, error, data refreshed).
- Added Nexus nav link to all 11 HTML files and updated nav.js router with initNexus import and page path mapping.
- Conducted visual QA of the complete Nexus page: verified dark canvas rendering, tooltip positioning, side panel scroll, frequency bar widths, and topic pill overflow.

Shivam Kumar — Graph Analyst

- Built the complete D3.js force-directed graph engine in frontend/js/nexus.js (1,121 lines): initSimulation() with forceLink/forceManyBody/forceCenter/forceCollide, ticked() for position updates, initZoom() with d3.zoom(), buildBaseGraph() for initial circular handle layout.
- Designed and implemented the 5-level node hierarchy: handle (white, radius by metric), hashtag (cyan #06B6D4), mention (purple #A855F7), link (orange #F97316), topic (green #22C55E). Defined COLORS constant and getNodeRadius() with log scale.
- Implemented expandHandle() with client-side cache, child node creation for top-10 entities across 4 categories, edge construction, simulation restart, and collapse logic.
- Implemented expandCategory() for granular per-category expansion triggered from the side panel category buttons, with expandedCategories Map for state tracking.
- Implemented all toolbar handlers: Show Labels (D3 text + rect elements), Node Size selector (updateNodeSizes() with simulation alpha restart), Topic filter (opacity transitions via D3 attr), Zoom controls (d3.zoom callbacks), Refresh Graph (cache clear + loadGraph() re-fetch).
- Integrated graph auto-refresh listener polling localStorage every 5 seconds for activeScrapperJob — triggers toast notification when graph_ready event detected.
- Verified COLORS object, node hierarchy, simulation parameters, and all D3 selections against live data rendering across all 20 handles.

WEEK 4 — TECHNICAL SUMMARY

Nexus Page — Architecture Overview

The Nexus is the most technically complex page in the linkmap platform. It combines D3.js force simulation, dual-database data merging, client-side caching, and real-time graph expansion into a single interactive interface.

Component	Technology	Detail
Graph Engine	D3.js v7	Force simulation: forceLink + forceManyBody + forceCenter + forceCollide
Node Hierarchy	5 levels	Handle (white) → Hashtag (cyan) → Mention (purple) → Link (orange) → Topic (green)
Data Source 1	PostgreSQL	7 SQL queries per handle: profile, hashtags, mentions, links, topics, stats, most liked tweet
Data Source 2	Neo4j	MENTIONED_BY_USER + SHARES_HASHTAG_WITH cross-handle relationships
Client Cache	JS Map()	handleCache + tweetCache prevent duplicate API calls on repeat node clicks
Expansion	On-demand	Child nodes added only when handle is clicked — base graph starts with 20 handle nodes only
Node Sizing	3 modes	By Followers (log scale), By Tweet Count (log scale), Equal (fixed 12px radius)
Topic Filter	D3 opacity	Matching handles opacity 1.0, non-matching 0.15, simulation unchanged
Zoom/Pan	d3.zoom()	Mouse wheel zoom, drag pan, programmatic zoom buttons, animated reset
Auto-refresh	localStorage poll	setInterval 5s checks for activeScraperJob, shows toast on graph_ready event
Neo4j Fallback	try/except	Returns empty lists if Neo4j offline — page renders with Postgres data only
Pool Integration	psycpg2 pool	All queries use pool.getconn()/putconn() — no raw psycpg2.connect() calls

Week 4 Deliverables Summary

Deliverable	File	Lines	Status
Nexus API — graph seed endpoint	backend/routes/nexus.py	86–157	✓
Nexus API — handle tweets endpoint	backend/routes/nexus.py	159–242	✓
Nexus API — handle details endpoint	backend/routes/nexus.py	244–462	✓
Nexus HTML — split-panel layout	frontend/nexus.html	~300	✓
Nexus JS — D3 force graph engine	frontend/js/nexus.js	1,121	✓
Graph toolbar — 5 control groups	frontend/js/nexus.js	inline	✓
Node expansion + collapse system	frontend/js/nexus.js	inline	✓
Side panel — 3 states + detail view	frontend/js/nexus.js	inline	✓
5-level node colour hierarchy	frontend/js/nexus.js	inline	✓
Client-side cache (Map)	frontend/js/nexus.js	inline	✓
Neo4j graceful fallback	backend/routes/nexus.py	47–84	✓
Nav update — all 11 HTML pages	frontend/*.html + nav.js	all	✓

Nexus Page — Description

The Nexus page is a full-viewport interactive graph explorer with a dark D3.js canvas on the left (70%) and an off-white intelligence detail panel on the right (30%). The graph initially renders all 20 scraped handles as white nodes sized by follower count. Clicking any handle node fetches its full details from the API and expands the graph with colour-coded child nodes representing its top hashtags (cyan), mentions (purple), links (orange), and topics (green). The side panel

simultaneously displays the handle's complete profile, engagement statistics, top hashtags and mentions with frequency bars, most liked tweet, and Neo4j cross-handle connections. The toolbar allows toggling labels, changing node size metric, filtering by topic, zooming, and refreshing the graph after new scrapes.

*Report prepared by: Rishav Pal (Data Engineer), Sayantan Mandal (OSINT Analyst), Shivam Kumar (Graph Analyst) | Week 4: 08
June – 14 June 2026*